

STUDY NOTES

Time Series, NLP, Deep Learning & Computer Vision

Everything from your photos, rewritten to be easy to understand. It covers the same topics, in the same depth, with the diagrams redrawn.

Contents

1. **Time Series Analysis:** what it is, why it's used, its 4 components, and 5 forecasting models
2. **NLP and Text Cleaning:** the 8 cleaning steps, the cleaning code, and a sentiment model
3. **Deep Learning Basics:** key terms, neural networks, learning types, network types, activation functions
4. **Neural Networks Explained** (Advanced AI Track slides): perceptron, gradient descent, loss, back-propagation, softmax, optimizers, CNN, RNN
5. **Deep Learning and Computer Vision:** TensorFlow/Keras/PyTorch, a first model, images as numbers, CNN layers, padding, stride, pooling, ResNet-50, YOLO, annotation, fine-tuning, segmentation

How to read this: The blue "In simple words" boxes explain an idea in everyday language. The yellow notes flag places where your photos were cut off or where two sources overlap. Code is shown exactly as in your photos.

Time Series Analysis

From: TCS slide deck "Time Series Analysis"

Agenda: 1. What is Time Series Analysis? · 2. Why Time Series Analysis? · 3. Components of Time Series · 4. Forecasting Models (ARIMA, SARIMA, ARIMAX, FARIMA, ARCH/GARCH)

1.1 What is Time Series Analysis?

Time Series Analysis is the study of data points recorded at successive time intervals, usually equally spaced, to understand patterns and forecast future values.

IN SIMPLE WORDS

A time series is a list of values recorded one after another over time: daily temperature, monthly sales, a stock's price every minute. Time series analysis looks at how the values moved in the past so we can guess where they go next.

Goal: the primary objectives are

- **Identify patterns** in historical data (trend, seasonality, cycles, irregularities).
- **Model relationships** within the data.
- **Forecast future values** based on past observations.

1.2 Why Time Series Analysis?

Time Series Analysis is essential because it helps organizations and researchers understand past patterns and predict future outcomes. The main reasons are:

1. Planning & Forecasting

Examples:

- Weather forecasting
- Economics & business

2. Performance Tracking

Example: compare yearly performance metrics (e.g., 2023, 2024, 2025, 2026) for:

- Sales growth
- Employee productivity
- Financial health

1.3 Components of Time Series

Any time series can be broken into four parts: **Trend**, **Seasonality**, **Cyclic** and **Irregularity**.

Component 1: Trend

Definition: A trend represents the long-term movement or direction in a time series. It shows whether the data is generally increasing, decreasing, or remaining constant over time.

Types of trend:

- **Uptrend:** values increase over time (e.g., population growth).
- **Downtrend:** values decrease over time (e.g., declining sales of outdated products).
- **Stationary/Horizontal:** no significant upward or downward movement; the data fluctuates around a constant mean.

Stationary vs Non-Stationary data

	Stationary	Non-Stationary
Mean and variance	Remain constant over time	Change over time
Modelling	Easier to model because patterns are stable	Common in real-world data; requires transformation (e.g., differencing) to make it stationary

IN SIMPLE WORDS

Mean is the average level of the data and **variance** is how widely it swings. If both stay the same over time, the data is **stationary**. **Differencing** means using the change from one point to the next (today – yesterday) instead of the raw values, which often removes a trend.

Component 2: Seasonality

Definition: Seasonality refers to patterns that repeat at regular, fixed intervals due to seasonal factors.

Examples:

- Retail sales spike during festivals or holidays.
- Electricity consumption rises in summer due to air conditioning.

Key characteristics:

- Occurs at predictable intervals (daily, weekly, monthly, yearly).
- Driven by calendar-related events (weather, holidays).

Component 3: Cyclic

Definition: Cyclic variations are long-term oscillations in data that occur over periods longer than a year, often linked to economic or business cycles.

Examples:

- Economic boom and recession cycles.

- Real estate market fluctuations.

Difference from Seasonality:

- Seasonality is **fixed and predictable** (e.g., every December).
- Cyclic patterns are **not fixed**; their duration and amplitude vary (e.g., 4–8 years).

Component 4: Irregularity

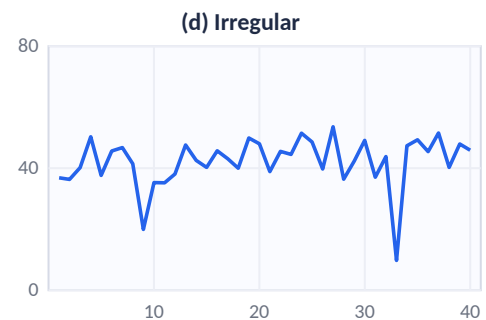
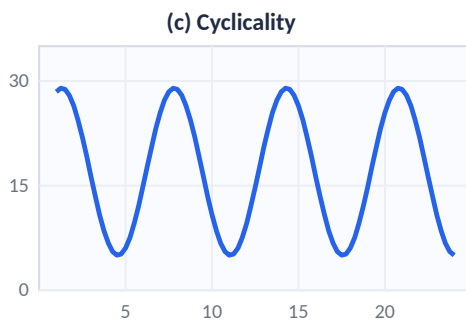
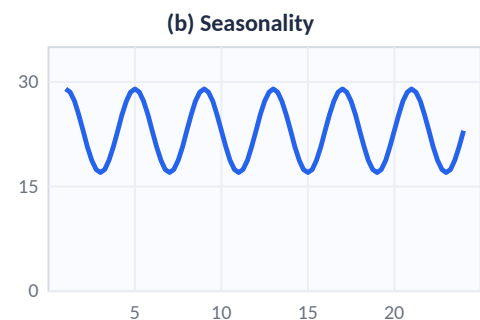
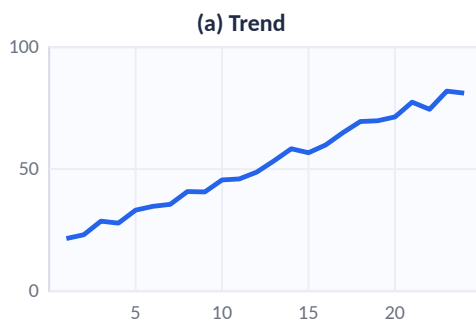
Definition: Irregular variations are random, unpredictable changes in data caused by unexpected events.

Examples:

- Natural disasters (floods, earthquakes).
- Pandemics (COVID-19).
- Sudden policy changes or strikes.

Key point: these variations are short-term and cannot be forecasted easily.

What the four components look like



(a) Trend climbs steadily. (b) Seasonality repeats the same shape at fixed intervals. (c) Cyclicality has big, long waves. (d) Irregular movement is random, with sudden drops.

Summary table

Component	Nature	Predictable?	Example
Trend	Long-term	Yes	GDP growth
Seasonality	Short-term	Yes	Holiday sales
Cyclic	Long-term	Partially	Economic cycles
Irregular	Random	No	Floods, COVID-19

1.4 Forecasting Models in Time Series

1. ARIMA (AutoRegressive Integrated Moving Average)

Purpose: ARIMA is one of the most widely used models for time series forecasting.

Assumptions:

- Data is **stationary** (constant mean and variance).
- No significant **outliers** are present.

Components:

Part	Name	What it means
AR (p)	Autoregression	The current value depends on its previous values.
I (d)	Integration	Differencing is applied to make the data stationary.
MA (q)	Moving Average	The current value depends on past forecast errors.

IN SIMPLE WORDS

ARIMA predicts the next value from (AR) the last few values, (MA) the last few mistakes it made, after first (I) differencing the data so it is stationary. **p, d and q** are the settings for how much of each to use.

2. SARIMA (Seasonal ARIMA)

Purpose: SARIMA extends ARIMA to handle seasonality in time series data.

Key feature: it adds seasonal components to ARIMA:

- **Seasonal AR (P):** autoregressive terms for seasonal lags.
- **Seasonal I (D):** seasonal differencing to remove seasonal trends.
- **Seasonal MA (Q):** moving average terms for seasonal errors.
- **Seasonal period (m):** length of the seasonal cycle (e.g., 12 for monthly data with yearly seasonality).

IN SIMPLE WORDS

Capital P, D and Q do the same jobs as small p, d and q, but they look back one whole season (for example, the same month last year) instead of one step.

3. ARIMAX (ARIMA with Exogenous Variables)

Purpose: ARIMAX is ARIMA plus external (exogenous) variables that influence the target variable.

Key feature: it incorporates predictors like marketing spend, economic indicators, or weather conditions.

Example: forecasting sales using past sales data *and* the advertising budget.

Use case: business forecasting where external factors matter.

4. FARIMA (Fractional ARIMA)

Purpose: FARIMA handles long-memory processes, where correlations decay slowly over time.

Key feature: it uses **fractional differencing** instead of integer differencing (as in ARIMA).

Why it's important: some time series (like climate or financial data) show persistent correlations that ARIMA cannot capture.

Use case: climate modelling, and financial time series with long-term dependencies.

IN SIMPLE WORDS

In ARIMA, **d** must be a whole number (1, 2...). In FARIMA it can be a fraction (like 0.4), so the model can "remember" events from much further back.

5. ARCH/GARCH (Volatility Models)

Purpose: models time-varying volatility in time series; commonly used in finance.

- **ARCH (Autoregressive Conditional Heteroskedasticity):** captures periods of high and low volatility.
- **GARCH (Generalized ARCH):** extends ARCH by including lagged variances and past errors for better volatility modelling.

Why it's important: financial markets often show **volatility clustering** (high volatility followed by high volatility).

Use case: stock price modelling, risk management, option pricing.

IN SIMPLE WORDS

Volatility means how wildly the values jump around. ARCH/GARCH doesn't just predict the value. It predicts how *shaky* the next period will be.

Comparison of the models

Model	Handles	Key Feature	Use Case
SARIMA	Trend + Seasonality	Seasonal ARIMA terms	Retail, weather
ARIMAX	Trend + External factors	Includes exogenous variables	Sales with marketing
FARIMA	Long memory	Fractional differencing	Climate, finance
ARCH/GARCH	Volatility	Time-varying variance	Stock prices

NLP and Text Cleaning

From: NLP.pdf (8 pages)

What NLP is, why raw text must be cleaned, the 8 cleaning steps with examples, the Python cleaning code, and a complete sentiment-analysis model.

2.1 What is NLP?

- NLP stands for **Natural Language Processing**. It is the field of AI that deals with understanding and generating human language: text and speech.
- NLP allows machines to read, interpret, and respond to language the way humans do.
- Common NLP tasks:

Task	Example question
Sentiment analysis	Is this review positive or negative?
Text classification	Is this email spam?
Named entity recognition	Find all person names in this document.
Machine translation	English to Hindi
Text summarisation	Make a short version of a long text.

2.2 Why Text Cleaning Is Necessary

- Raw text from the internet, customer reviews, or social media is **messy**.
- It contains punctuation, HTML tags, URLs, numbers, special characters, capital letters, and filler words that add no meaning.
- If you feed this raw text directly to a model, it will treat '**Running**', '**running**' and '**RUNNING**' as three different words. It will also treat '**good!**' and '**good**' as different.

IN SIMPLE WORDS

Cleaning makes sure the same word always looks the same, and throws away the parts that carry no meaning, so the model can focus on the words that matter.

2.3 The 8 Text Cleaning Steps

1 Lowercasing

Convert all characters to lowercase. This ensures 'Apple', 'apple' and 'APPLE' are treated as the same word. It is almost always the first step.

BEFORE

The Movie Was AMAZING and I Loved It

AFTER

the movie was amazing and i loved it

2 Removing HTML Tags

If text is scraped from websites it will contain HTML markup. Tags like `<br>`, `<p>`, `<a href=...>` are not meaningful for NLP and must be removed.

BEFORE

`<p>This is a <b>great</b> product!</p>`

AFTER

This is a great product!

3 Removing URLs

Hyperlinks like `https://www.example.com` add no semantic meaning to the text. They are removed using a **regular expression** (a text pattern) that matches the URL pattern.

BEFORE

Check this out `https://www.example.com`
for more details

AFTER

Check this out for more details

4 Removing Punctuation and Special Characters

Punctuation marks like full stops, commas and exclamation marks, and special characters like `@` `#` `$` `%`, are removed. They add little meaning in most classification tasks.

BEFORE

Wow!!! This product is #1 and it's
amazing @user

AFTER

Wow This product is 1 and its amazing
user

5 Removing Numbers

Numbers are often removed unless the task specifically requires them (for example, financial text analysis). In most text classification tasks numbers do not carry meaning.

BEFORE

not visible in your photos

AFTER

I ordered items on th January

6 Removing Stop Words

Stop words are very common words that appear in almost every sentence but carry little meaning by themselves, like 'the', 'is', 'a', 'an', 'in', 'on', 'and', 'but', 'for'. Removing them reduces the vocabulary size and helps the model focus on meaningful words.

BEFORE

the movie was really good and i loved it
a lot

AFTER

movie really good loved lot

7 Stemming

Stemming reduces words to their root form by **chopping off suffixes using fixed rules**. The result may not always be a real word, but it groups related forms together.

```
running, runs, runner → run
playing, played, player → play
```

8 Lemmatisation

Lemmatisation also reduces words to their root form, but it uses **vocabulary and grammar rules**, so the result is always a real dictionary word (the **lemma**).

```
running → run
better → good
studies → study
was → be
```

- For most tasks **lemmatisation is preferred**.
- **Stemming is faster** and acceptable when speed matters more than accuracy.

	Stemming	Lemmatisation
How	Chops endings off with fixed rules	Looks the word up using vocabulary and grammar
Result	May not be a real word	Always a real dictionary word
Speed	Faster	Slower
When to use	When speed matters more than accuracy	Most tasks (preferred)

Note: Your photos were cut off in two places here: the **Before** sentence for Step 5, and anything after "was → be" in the Step 8 examples.

2.4 Text Cleaning Code (Python + NLTK)

This code runs all the steps above on some example reviews. **NLTK** is a Python library for working with language, and **re** is Python's regular-expression library.

```
# Install & download (run once)
import nltk
import re
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')

# Import libraries
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, PorterStemmer

# Initialize tools
stop_words = set(stopwords.words('english'))
```

```

lemmatizer = WordNetLemmatizer()
stemmer = PorterStemmer()

# Text cleaning function
def clean_text(text):
    # Convert text to lowercase
    text = text.lower()
    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)
    # [line(s) here were hidden between two photos - most likely URL removal:]
    # text = re.sub(r'http\S+|www\.S+', '', text)
    # Remove special characters & numbers
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenization
    tokens = text.split()
    # Remove stopwords
    tokens = [w for w in tokens if w not in stop_words]
    # Lemmatization
    tokens = [lemmatizer.lemmatize(w) for w in tokens]
    # OPTIONAL: Stemming (uncomment if needed)
    #tokens = [stemmer.stem(w) for w in tokens]
    # Join words back into string
    return ' '.join(tokens)

#Example dataset
sample_reviews = [
    'The movie was AMAZING!!! I absolutely Loved it. Check www.movies.com',
    '<b>Worst</b> product ever. Do NOT buy this. Totally 0/10 rating.',
    'I have been using this for 3 months and it is running perfectly fine!',
    'Highly recommended! The customer service was very helpful and polite.',
]

#Run and display results
import pandas as pd
df = pd.DataFrame({'Original': sample_reviews})

```

What each part does:

Code	What it does
<code>nltk.download(...)</code>	Downloads the stop-word list, the WordNet dictionary (used by the lemmatiser) and the tokenizer data. You only need to run this once.
<code>re.sub(r'<.*?>', '', text)</code>	Finds anything between < and > (an HTML tag) and replaces it with nothing.
<code>re.sub(r'[^a-z\s]', '', text)</code>	Removes every character that is <i>not</i> a lowercase letter or a space, so punctuation, symbols and numbers all go.
<code>text.split()</code>	Tokenization: splits the sentence into a list of single words (tokens).
<code>[w for w in tokens if w not in stop_words]</code>	Keeps only the words that are not stop words.
<code>lemmatizer.lemmatize(w)</code>	Turns each word into its dictionary form.
<code>' '.join(tokens)</code>	Glues the words back into one string with spaces.

Note: One or two lines of code between **Remove HTML tags** and **Remove special characters** fell between two of your photos. The commented-out line shown in the code is the URL-removal line used in the same document's model code (section 2.5), and it's almost certainly what's missing.

2.5 A Complete NLP Model (Sentiment Analysis)

Clean text



Convert to numbers



Train model



Predict

IN SIMPLE WORDS

A model can't read words, only numbers. So we (1) clean each review, (2) turn it into numbers with **TF-IDF**, (3) train a **Logistic Regression** model on reviews already labelled positive (1) or negative (0), and (4) ask it to label new sentences.

```
#Download NLTK data
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')

#Imports
import re
import nltk
import pandas as pd
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```

#Initialize tools
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

# Text cleaning function
def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'http\S+|www\.\S+', '', text)
    tokens = text.split()
    tokens = [w for w in tokens if w not in stop_words]
    tokens = [lemmatizer.lemmatize(w) for w in tokens]
    return ' '.join(tokens)

#Sample dataset (text + labels)
data = {
    'review': [
        'I love this product, it is amazing!',
        'Worst experience ever, very bad service.',
        'Absolutely fantastic, highly recommend!',
        'Not good, very disappointing.',
        'Best purchase I have made!',
        'Terrible quality, waste of money.'
    ],
    'sentiment': [1, 0, 1, 0, 1, 0] # 1 = Positive, 0 = Negative
}
df = pd.DataFrame(data)

#Clean the text
df['cleaned'] = df['review'].apply(clean_text)

# Convert text to numbers (TF-IDF)
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['cleaned'])
y = df['sentiment']

# [start of this statement was hidden between two photos:]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

#Train model
model = LogisticRegression()
model.fit(X_train, y_train)

#Predictions
y_pred = model.predict(X_test)

#Accuracy
print("Model Accuracy:", accuracy_score(y_test, y_pred))

#Test with new sentence
test_sentence = "This product is really amazing and useful!"
cleaned_test = clean_text(test_sentence)
test_vector = vectorizer.transform([cleaned_test])
prediction = model.predict(test_vector)
print("\nTest Sentence:", test_sentence)
print("Prediction:", "Positive" if prediction[0] == 1 else "Negative")

```

The key new ideas in this code:

Term	Meaning
TF-IDF (<code>TfidfVectorizer</code>)	Turns each cleaned review into a row of numbers. A word gets a high score if it appears often in <i>this</i> review but rarely across all reviews, so the numbers show which words matter most.
<code>fit_transform</code> vs <code>transform</code>	<code>fit_transform</code> learns the vocabulary from the training text and converts it. <code>transform</code> converts new text using that same vocabulary, which is why the test sentence uses only <code>transform</code> .
train_test_split	Splits the data. Here 80% is used for training and 20% (<code>test_size=0.2</code>) is kept aside to test the model. <code>random_state=42</code> makes the split the same every run.
Logistic Regression	A simple classification model that learns to output 1 (positive) or 0 (negative).
<code>accuracy_score</code>	The fraction of test reviews the model labelled correctly.

Deep Learning Basics

From: Deeplearning.pdf (6 pages)

What deep learning is, when to use it, where it's used, the key terms, how neural networks are built, how they learn, the main network types, and the four common activation functions.

3.1 What is Deep Learning?

Deep Learning is a branch of Artificial Intelligence (AI) that enables machines to learn patterns from large amounts of data using **multi-layered neural networks**. It is widely used in image recognition, speech processing and natural language understanding.

When to use Deep Learning

- **Large amounts of data are available:** deep learning needs a lot of examples to learn well.
- **The input is unstructured:** images, text, audio, video.
- **Traditional ML has plateaued** and more performance is needed.
- **Enough computing power is available:** GPUs are strongly recommended.

Key applications

- **Computer Vision:** image classification, object detection, facial recognition
- **Natural Language Processing:** text classification, translation, chatbots, summarisation
- **Speech Recognition:** voice assistants, transcription
- **Recommendation Systems:** Netflix, YouTube, Amazon product suggestions
- **Healthcare:** tumour detection in X-rays, drug discovery
- **Generative AI:** creating images, writing text, generating code

3.2 Important Terms to Know

Term	Meaning
Neuron	The basic unit. It receives inputs, multiplies each by a weight, adds a bias term, and passes the result through an activation function to produce an output.
Layer	A collection of neurons. All neurons in a layer receive inputs from the previous layer and pass outputs to the next layer.
Weight	A number attached to each connection between neurons. During training these are adjusted so the model learns the correct mapping from inputs to outputs.
Bias	An extra adjustable value added to each neuron. It shifts the activation function so the model can fit patterns that do not pass through the origin.
Activation Function	A mathematical function applied to the output of each neuron. Without it every layer would just be a linear transformation, and stacking many layers would give no extra power. Activation functions introduce non-linearity so the network can learn complex patterns.
Epoch	One complete pass through the entire training dataset. Training typically runs for many epochs until the model converges.
Batch Size	The number of training examples used in one parameter update step. Smaller batches give noisier updates but use less memory. Common values are 32, 64, 128.
Loss Function	Measures how wrong the model's predictions are. The goal of training is to minimise the loss. For regression use Mean Squared Error . For binary classification use Binary Cross-Entropy . For multi-class classification use Categorical Cross-Entropy .
Optimiser	The algorithm that updates weights to minimise the loss. The most popular is Adam , which adapts the learning rate for each parameter automatically. SGD (Stochastic Gradient Descent) is the classic option.

IN SIMPLE WORDS

A neuron does: **output = activation(input₁×weight₁ + input₂×weight₂ + ... + bias)**. Training means repeatedly nudging the weights and biases so the loss gets smaller. The optimiser decides *how* to nudge them.

3.3 Introduction to Neural Networks

Neural networks are machine learning models that mimic the complex functions of the human brain. They consist of interconnected nodes, or neurons, that process data, learn patterns, and enable tasks such as pattern recognition and decision making. Neural networks can learn and identify patterns **directly from data, without pre-defined rules**.

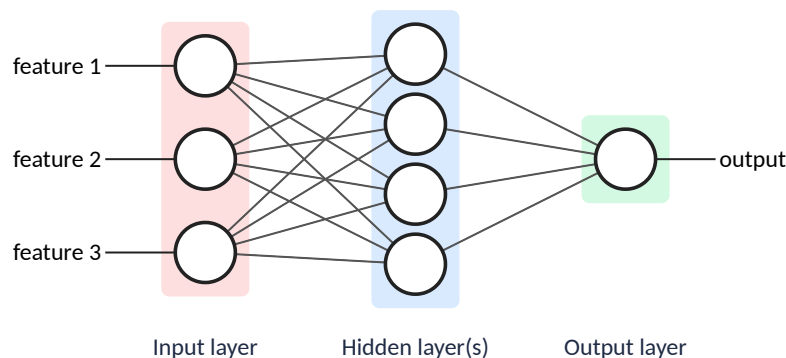
These networks are built from several key components:

- **Neurons:** the basic units that receive inputs. Each neuron is governed by a threshold and an activation function.
- **Connections:** links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases:** these parameters determine the strength and influence of the connections.
- **Propagation Functions:** mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule:** the method that adjusts weights and biases over time to improve accuracy.

Importance of Neural Networks

- **Identify complex patterns:** recognise intricate structures and relationships in data; adapt to dynamic and changing environments.
- **Learn from data:** handle vast datasets efficiently; improve performance with experience and retraining.
- **Drive key technologies:** power natural language processing (NLP); enable self-driving vehicles; support automated decision-making systems.
- **Boost efficiency:** streamline workflows and processes; enhance productivity across industries.
- **Backbone of AI:** serve as the core driver of artificial intelligence progress; continue shaping the future of technology and innovation.

Layers in Neural Network Architecture



1. **Input Layer:** this is where the network receives its input data. Each input neuron in the layer corresponds to a **feature** in the input data.
2. **Hidden Layers:** these layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something the output layer can use.
3. **Output Layer:** the final layer produces the output of the model. The format of these outputs varies depending on the specific task, such as classification or regression.

3.4 Learning of a Neural Network

1. Supervised Learning

A neural network learns from **labelled input-output pairs** provided by a "teacher". The network generates outputs based on inputs, and by comparing these outputs to the known desired outputs, an **error signal** is created. The network iteratively adjusts its parameters to minimise errors until it reaches an acceptable performance level.

2. Unsupervised Learning

Involves data **without labelled output variables**. The primary goal is to understand the underlying structure of the input data (X). Unlike supervised learning, there is no instructor to guide the process. Instead, the focus is on modelling data patterns and relationships, with techniques like **clustering** and **association** commonly used.

IN SIMPLE WORDS

Supervised means you have the answers ("this photo is a cat") and the network learns to match them.

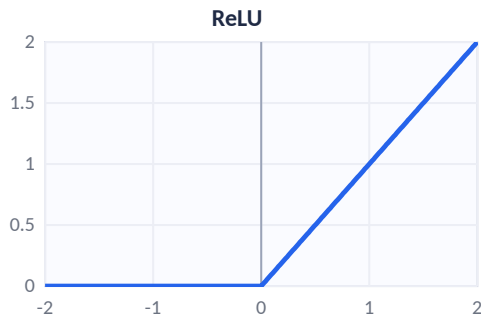
Unsupervised means there are no answers, so the network finds groups or patterns on its own.

3.5 Types of Neural Networks

Type	What it is
Feedforward Network	A simple artificial neural network architecture in which data moves from input to output in a single direction.
Single-layer Perceptron	Has one layer; it applies weights, sums the inputs and uses an activation to produce the output.
Multilayer Perceptron (MLP)	A type of feedforward neural network with three or more layers: an input layer, one or more hidden layers and an output layer.
Convolutional Neural Network (CNN)	Designed for image processing. It uses convolutional layers to automatically learn features from input images, enabling effective image recognition and classification.
Recurrent Neural Network (RNN)	Handles sequential data using feedback loops to retain context over time.
Long Short-Term Memory (LSTM)	A type of RNN with memory cells and gates to handle long-term dependencies and avoid vanishing gradients.

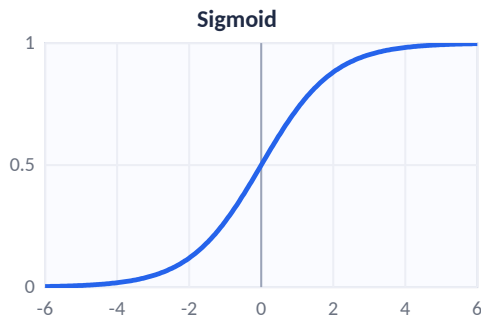
Diagrams of the perceptron, MLP, CNN and RNN are in Part 4. CNNs are covered in depth in Part 5.

3.6 Activation Functions



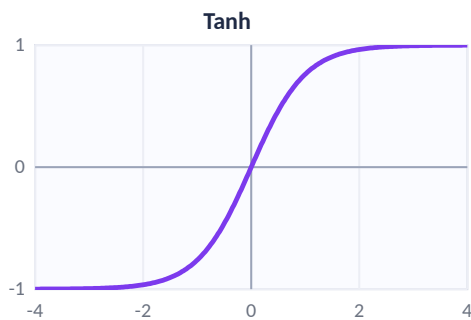
ReLU (Rectified Linear Unit)

Output = $\max(0, x)$. If the input is positive, pass it through. If it is negative, output zero. It is the most common choice for hidden layers because it is simple, fast to compute, and avoids the vanishing gradient problem.



Sigmoid

Output = $1 / (1 + e^{-x})$ ("1 divided by 1 plus e to the power of minus x"). It squashes any value into a range between 0 and 1. It is used in the **output layer for binary classification** to give a probability.



Tanh

Output ranges from **-1 to +1**. It is zero-centred, unlike sigmoid. It is sometimes used in hidden layers, but ReLU is generally preferred.

2.0	1.0	0.1
-----	-----	-----

softmax ↓

0.66	0.24	0.10
------	------	------

raw scores → probabilities that add up to 1

Softmax

Converts a vector of raw scores into probabilities that **sum to 1**. It is used in the **output layer for multi-class classification**. Each output neuron gives the probability of that class.

Note: The softmax example numbers (2.0, 1.0, 0.1 → 0.66, 0.24, 0.10) are a worked illustration I added. They are not from your photos. The formula and properties of softmax are in section 4.8.

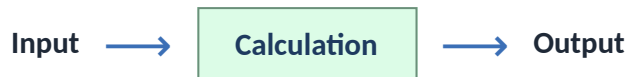
Neural Networks Explained

From: TCS "Advanced AI Track" slides 2-20

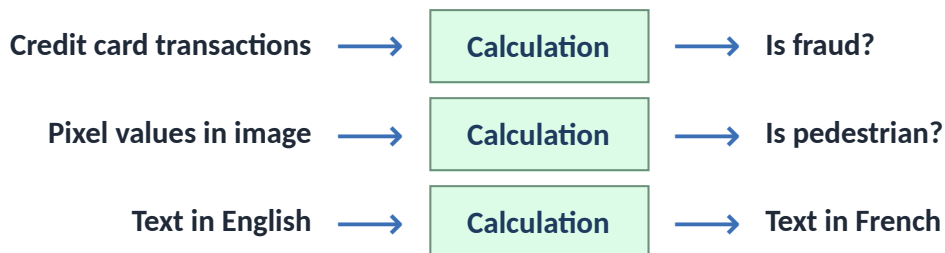
How a network actually works: the basic structure, why we need neural networks, the perceptron, activation functions, gradient descent and its problems, loss functions, back-propagation, softmax, output layers, optimisers (SGD, RMSprop, Adam), FCN/FNN, convolution, stride, padding, pooling, CNN architecture and RNN.

4.1 Simple Deep Learning Structure

Every deep learning model has the same basic shape: some **input** goes in, a **calculation** happens, and an **output** comes out.

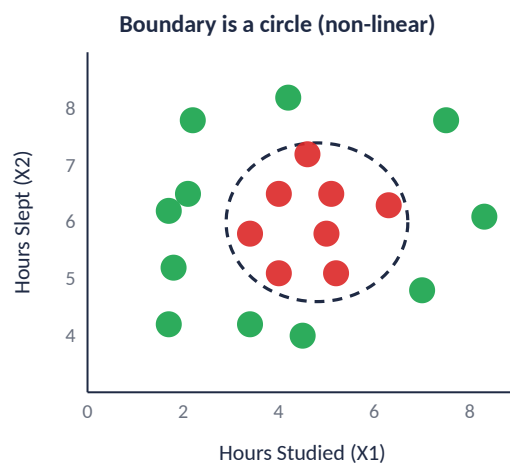
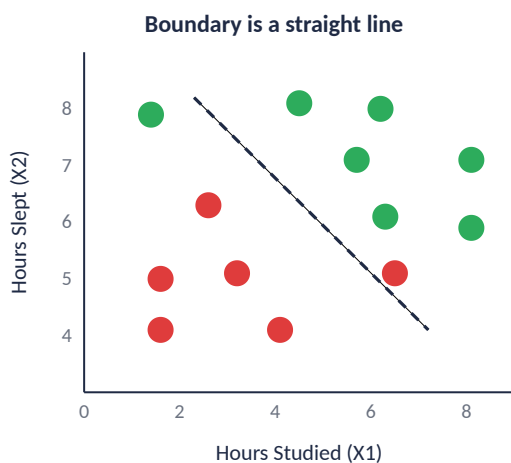
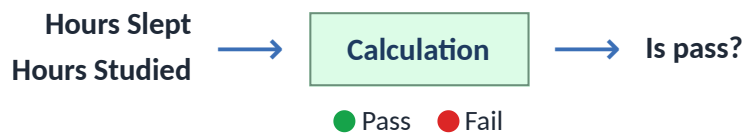


Examples:



4.2 Why Artificial Neural Network?

Goal: predict whether a student passes an exam, using two inputs: **Hours Studied (X1)** and **Hours Slept (X2)**.



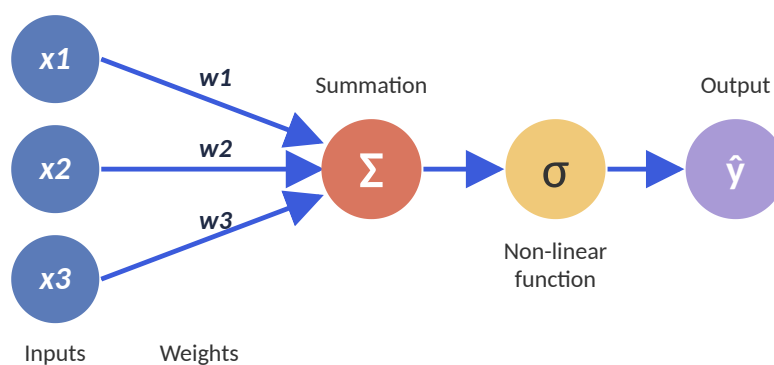
The **first problem** (left) could be solved by **Logistic Regression**, because a straight line separates pass from fail. But when the separation boundary is **non-linear** (right: the fails sit inside a circle) and we do not have this information in advance (*a priori*), an **ANN may be a solution**.

IN SIMPLE WORDS

Simple models can only draw straight dividing lines. A neural network can learn curved, complicated boundaries by itself, without being told the shape beforehand.

4.3 Perceptron and Multi-Layered Perceptron

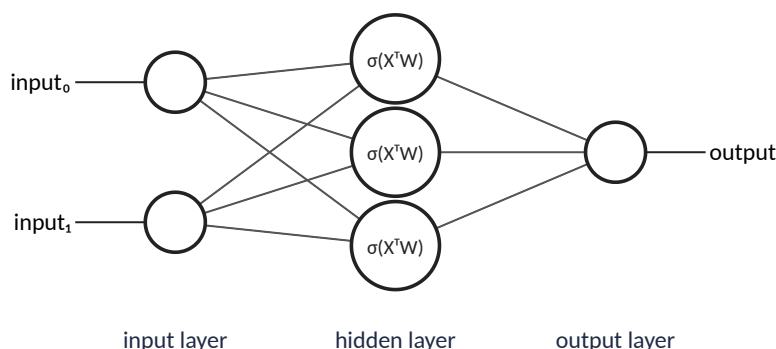
Perceptron: the basic unit of an ANN. Inputs x_1, x_2, x_3 are multiplied by their weights w_1, w_2, w_3 , added together (Σ , **summation**), then passed through a **non-linear function** σ to give the output $\hat{y}$.



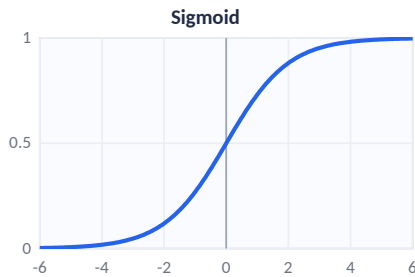
$$\hat{y} = \sigma(X^T W)$$

$X^T W$ is just "multiply every input by its weight and add them up" ($x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3$). σ is the activation function, and $\hat{y}$ ("y-hat") is the predicted output.

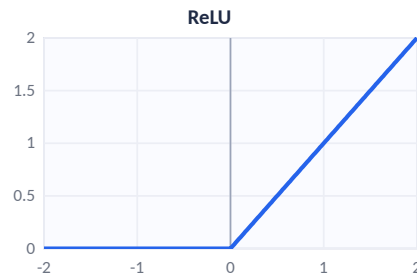
Multi-Layered Perceptron (MLP): many perceptrons arranged in layers. Each hidden neuron computes its own $\sigma(X^T W)$.



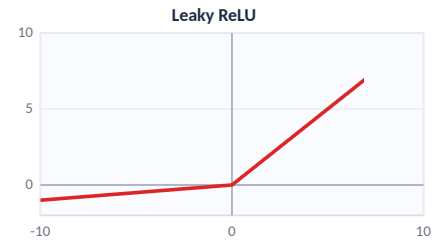
4.4 Activation Functions



Sigmoid function



ReLU function



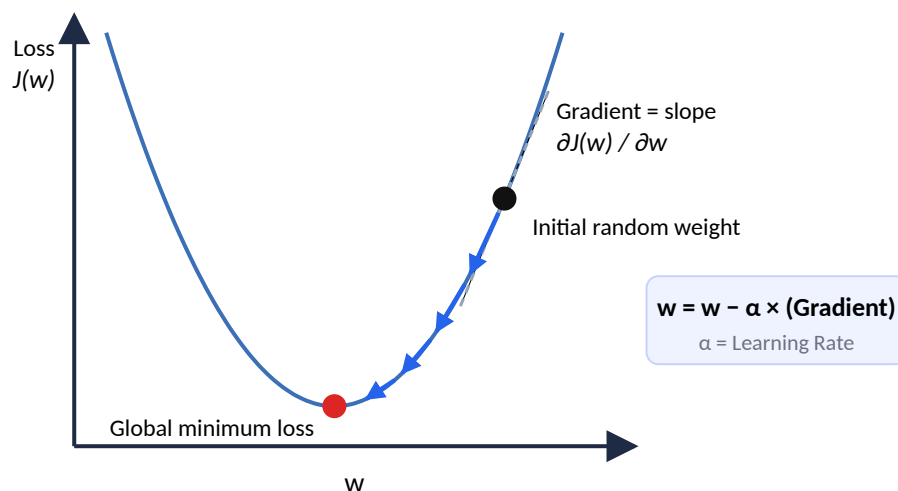
Leaky ReLU (negative side drawn steeper to be visible)

$$\text{Leaky ReLU: } f(x) = 0.01x \text{ for } x < 0 \quad | \quad f(x) = x \text{ for } x \geq 0$$

IN SIMPLE WORDS

Leaky ReLU is ReLU with a tiny slope (0.01) for negative inputs instead of a flat zero, so neurons with negative inputs still pass on a small signal and don't "die". Sigmoid, ReLU and Tanh are explained in section 3.6.

4.5 Gradient Descent



- The curve shows **Loss $J(w)$** for every possible value of a weight **w** . We want the weight at the bottom: the **global minimum loss**.
- We start at an **initial random weight**.
- The **gradient**, $\partial J(w) / \partial w$, is the slope of the curve at that point. It tells us which direction is "uphill".
- Update rule: **$w = w - \alpha \times (\text{Gradient})$** , where **$\alpha$ is the learning rate** (how big a step to take). The minus sign moves us *downhill*.
- This process is repeated in a loop many times, and the iterations are called **epochs**.

IN SIMPLE WORDS

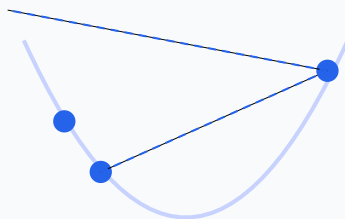
Imagine standing on a foggy hill and wanting to reach the valley. You feel which way the ground slopes (the gradient) and take a step downhill. The size of the step is the learning rate. You keep repeating until you reach the bottom.

4.6 Gradient Descent: Problems



Vanishing Gradient

The steps get smaller and smaller, so the weights almost stop changing and learning stalls.



Exploding Gradient

The steps get far too big, so the weights jump wildly and never settle.



Trapped in Local Minima

The descent stops in a small dip and never reaches the real lowest point.

4.7 Loss Functions

For each sample, **error** = $\hat{y} - y$ (prediction minus the true answer):

Sample	$\hat{y}$ (predicted)	y (true)	error
X_1	0.9	1	-0.1
X_2	0.2	0	0.2
X_3	0.1	1	-0.9
X_4	0.5	0	0.5

Mean Squared Error (MSE)

$$\mathcal{L} = \frac{1}{2} (\hat{y} - y)^2$$

Used for **continuous data**.

Cross Entropy (logistic)

$$\mathcal{L} = -(y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}))$$

Used for **categorical data**.

Cost function: the average loss over all n samples:

$$J = (1/n) \sum_{i=1 \dots n} \mathcal{L}(\hat{y}_i, y_i)$$

Goal: find the set of weights that minimises the losses.

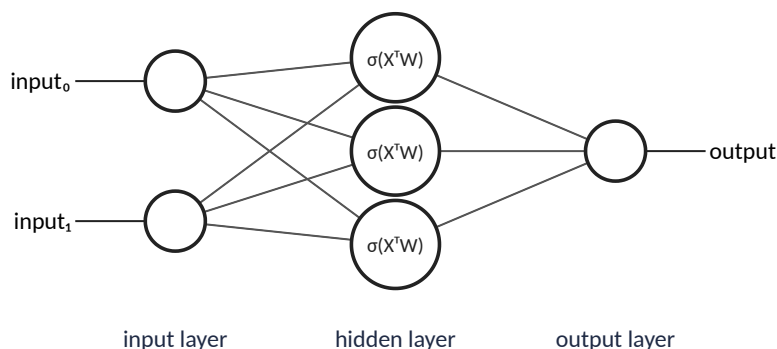
IN SIMPLE WORDS

Loss is how wrong the model is on *one* example. **Cost** is the average loss over *all* examples. Training tries to make the cost as small as possible.

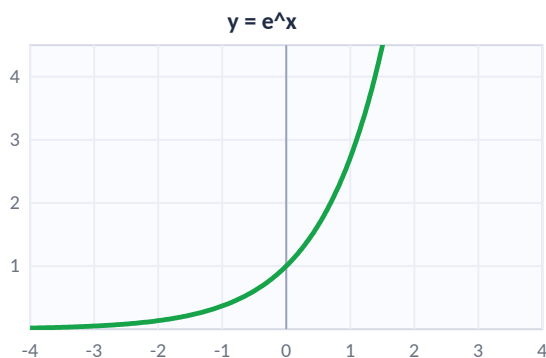
4.8 Forward and Backward Propagation

- **Forward propagation:** compute the output based on the input (data flows left → right).
- **Backward propagation:** adjust the weights based on the loss/cost (the error flows right → left).

Backward propagation is like gradient descent.



4.9 Softmax Function



$e = 2.718...$

The function e^x is **always positive**, even for negative x (look at the curve: it never goes below 0).

$$\sigma_i = e^{z_i} / \sum e^z$$

Take e to the power of each score z_i , then divide by the total of all of them.

Properties:

1. The sum of the outputs is always 1.
2. Values are always positive.
3. They can be treated as probabilities, but they are not exactly probabilities.
4. These properties make them suitable for deep learning applications, especially in **multi-class classification**.

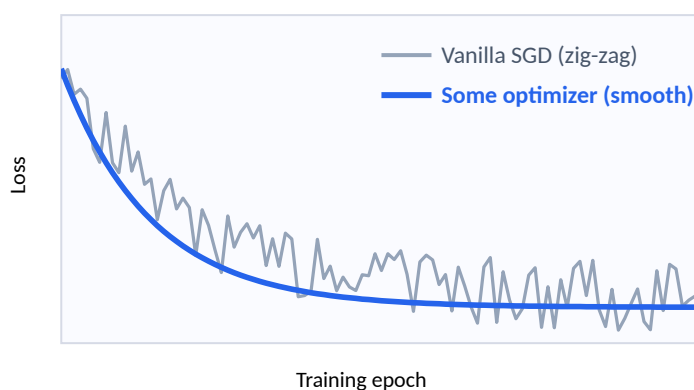
4.10 Output Layer Architecture

Task	Output layer	Loss function
Predict continuous data	One unit with a linear activation function	MSE
Binary classification	One unit with a sigmoid activation function	Binary Cross Entropy
Multiclass (N-way) classification	N units with a softmax activation function	Cross Entropy

4.11 Optimizer

An **optimizer** is the algorithm that adjusts the weights during backward propagation.

Optimizers are simply modifications of **SGD (Stochastic Gradient Descent)**, with the goal of smoothing the descent.



Note: On the slide, the two curves in this graph were missing (probably an animation). This is a redrawn version of what such a graph normally shows: vanilla SGD zig-zags down, while a better optimizer comes down smoothly.

RMSprop Optimizer

$$v = (1 - \beta)(dJ)^2 + \beta \cdot v_{t-1}$$

$$w \leftarrow w - \left(\eta / \sqrt{v + \epsilon} \right) \cdot dJ$$

Key point: the step size for adjusting the weight depends on the **history of the gradient magnitudes**.

- Large gradient → smaller steps
- Small gradient → larger steps

Large gradients indicate oscillations or volatile learning.

IN SIMPLE WORDS

v is a running average of how big the recent gradients were. Dividing the step by $\sqrt{v}$ means that if the gradients have been big and jumpy, the step shrinks, and if they've been small, the step grows. η is the learning rate. ϵ is a tiny number that stops division by zero.

Adam Optimizer

$$v = (1 - \beta_1) \cdot dJ + \beta_1 \cdot v_{t-1}$$

$$s = (1 - \beta_2)(dJ)^2 + \beta_2 \cdot s_{t-1}$$

$$w \leftarrow w - (\eta / \sqrt{(\tilde{s} + \epsilon)}) \cdot \tilde{v}$$

Recommended parameters

$\eta = 0.001$ (learning rate)

$\beta_1 = 0.9$

$\beta_2 = 0.999$

$\epsilon = 10^{-8}$

Key point: Adam (Adaptive Momentum) combines both **momentum** and **RMSprop**.

- $v \rightarrow$ from momentum (the running average of the gradient itself: it keeps moving in a consistent direction)
- $s \rightarrow$ from RMSprop (the running average of the squared gradient: it controls the step size)

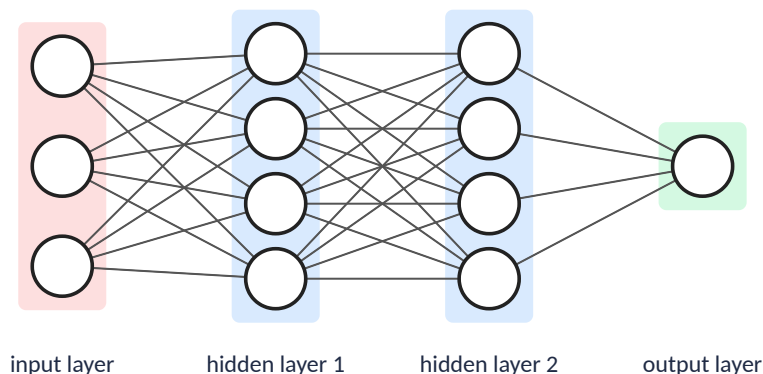
Bias correction terms:

$$\tilde{v} = v / (1 - \beta_1^t) \quad \tilde{s} = s / (1 - \beta_2^t)$$

IN SIMPLE WORDS

At the start, v and s begin at zero, so their early values are too small. Dividing by $(1 - \beta^t)$ fixes this. The correction is large in the first few steps and fades away as t (the step number) grows.

4.12 FCN / FNN



- **Fully Connected Neural Network (FCN):** "fully connected" means each node is connected to all the nodes of the next layer.
- **Feed Forward Neural Network (FNN):** "feedforward" means that the activation propagates from input to output.



4.13 Convolution: Introduction

A small grid of numbers called a **kernel** (or filter) slides over the image. At each position, multiply the overlapping numbers and add them up. That sum becomes one cell of the result.

Image						Kernel						Result				
0	1	0	0	1	*	1	1	1	=							
0	1	0	0	1		-1	-1	-1			0	0	0			
0	1	1	1	1		0	0	0			-1	-2	-2			
0	1	0	0	1							1	2	2			
0	1	0	0	1												

Worked example (top-left position, shaded): the kernel sits on the top-left 3×3 of the image.

Row 1: $(0 \times 1) + (1 \times 1) + (0 \times 1) = 1$ · Row 2: $(0 \times -1) + (1 \times -1) + (0 \times -1) = -1$ · Row 3: everything $\times 0 = 0 \rightarrow 1 - 1 + 0 = 0$

This kernel compares each row with the row below it. The result is non-zero only where the image changes from top to bottom (the horizontal bar in the middle row). So it **detects horizontal edges**.

Note: The worked calculation and the "detects horizontal edges" explanation were added by me. The image, kernel and result values are exactly as on the slide.

4.14 Stride and Padding

Padding: a border of zeros added around the image

0	0	0	0	0	0	0
0	0	1	0	0	1	0
0	0	1	0	0	1	0
0	0	1	1	1	1	0
0	0	1	0	0	1	0
0	0	1	0	0	1	0
0	0	0	0	0	0	0

Stride: how far the window jumps each move

0	1	0	0	1
0	1	0	0	1
0	1	1	1	1
0	1	0	0	1
0	1	0	0	1

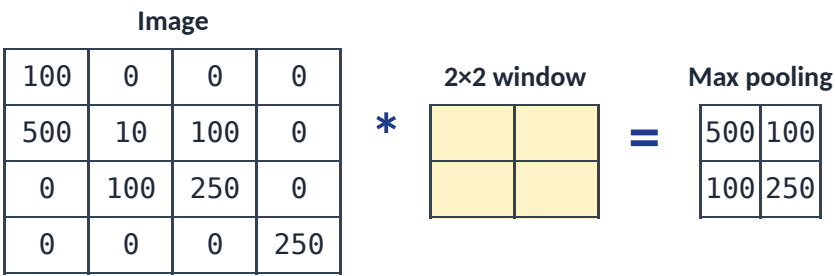
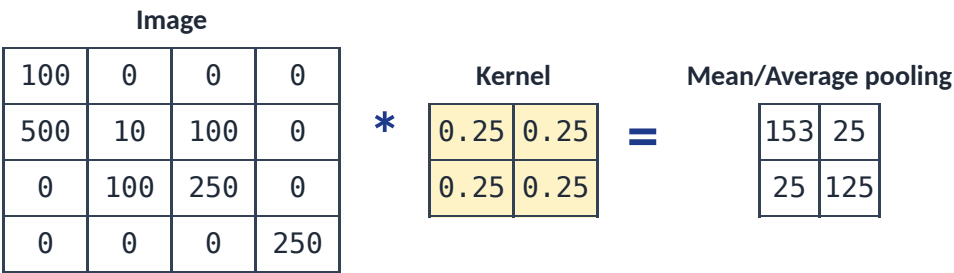
stride 2 → window moves 2 columns right

0	1	0	0	1
0	1	0	0	1
0	1	1	1	1
0	1	0	0	1
0	1	0	0	1

Padding and stride are explained fully, with the output-size formula, in section 5.8.

4.15 Pooling

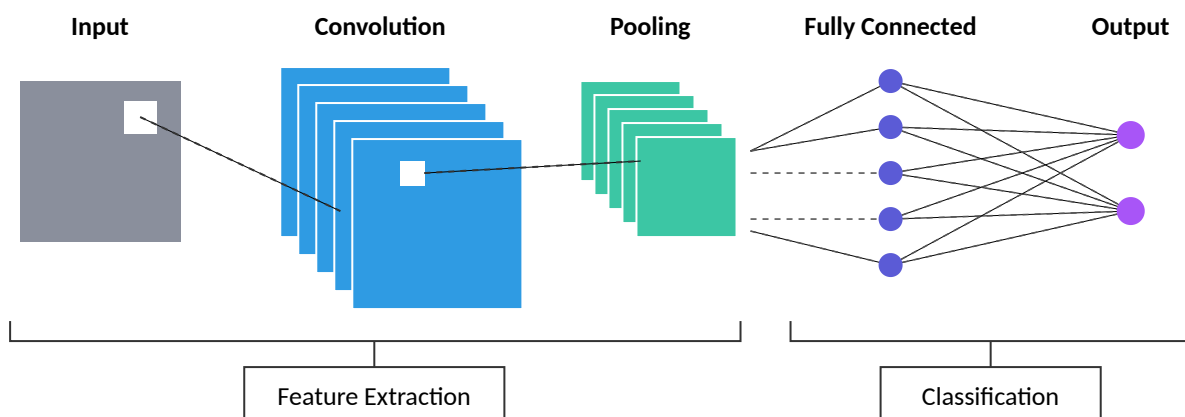
Stride and pooling are both methods to down-sample (make the image smaller).



The image is split into four 2x2 blocks. Each block becomes one number:

Block	Values	Average (× 0.25 each)	Max
Top-left	100, 0, 500, 10	$610 \div 4 = 152.5 \approx \mathbf{153}$	500
Top-right	0, 0, 100, 0	$100 \div 4 = \mathbf{25}$	100
Bottom-left	0, 100, 0, 0	$100 \div 4 = \mathbf{25}$	100
Bottom-right	250, 0, 0, 250	$500 \div 4 = \mathbf{125}$	250

4.16 Convolutional Neural Network: Architecture

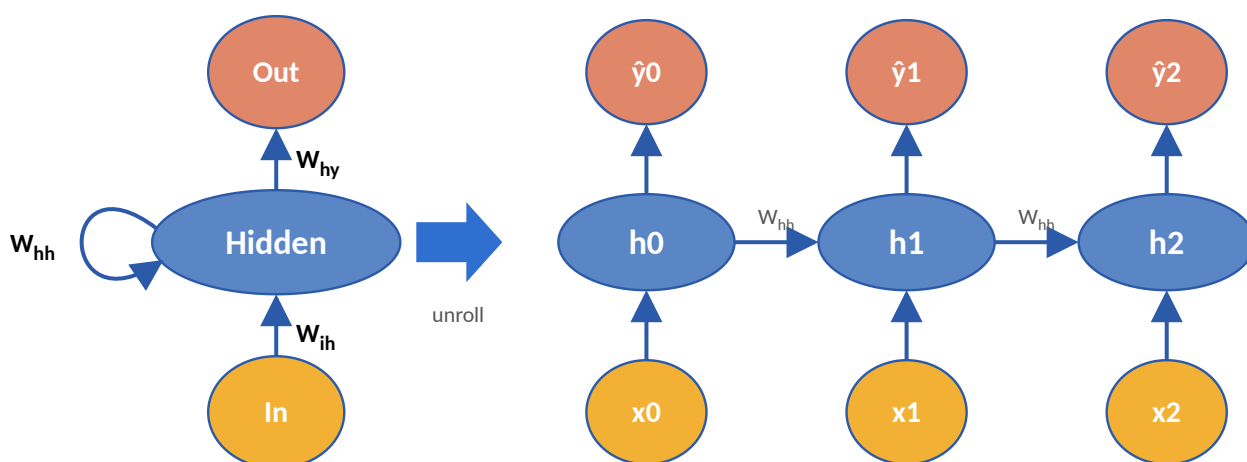


- **Feature extraction** (convolution + pooling): finds edges, shapes and patterns in the image and shrinks it down.
- **Classification** (fully connected + output): uses those features to make the final decision.



4.17 Recurrent Neural Network (RNN)

Recurrent Neural Networks (RNNs) are specialised neural networks designed for **sequential and time-dependent data**.



- **Left (folded)**: the hidden layer feeds its own output back into itself through the loop W_{hh} . That loop is its memory.
- **Right (unrolled over time)**: the same network is used at each step. At step t it takes the new input x_t (through W_{ih}) and the previous hidden state h_{t-1} , and produces output $\hat{y}_t$ (through W_{hy}).
- W_{ih} = input→hidden weights, W_{hh} = hidden→hidden weights, W_{hy} = hidden→output weights. The same three weight sets are reused at every time step.



Deep Learning and Computer Vision

From: CVdocx.pdf (19 pages)

The tools (TensorFlow, Keras, PyTorch), tensors, a first Keras model, how computers see images, preparing images, data augmentation, CNN layers, convolution filters, padding, stride and pooling, ResNet-50 and YOLO, annotation, fine-tuning, classification and segmentation.

5.1 TensorFlow, Keras and PyTorch

Name	What it is	Made by
TensorFlow	The full deep learning engine. Handles maths, GPU and deployment.	Google
Keras	The easy interface that sits on top of TensorFlow. This is what we write.	Google
PyTorch	A competing framework that does the same job in a more Python-like way.	Meta

IN SIMPLE WORDS

Simple way to remember it: TensorFlow is the engine of a car, Keras is the steering wheel, and PyTorch is a different car that works just as well.

Installing and checking

```
pip install tensorflow
pip install torch torchvision
pip install opencv-python matplotlib
```

```
import tensorflow as tf
import torch

print('TensorFlow :', tf.__version__)
print('PyTorch    :', torch.__version__)
```

OUTPUT

```
TensorFlow : 2.15.0
PyTorch    : 2.1.0
```

5.2 Tensor: the basic data type

A **tensor** is simply an array of numbers, just like a NumPy array. The only difference is that a tensor can run on a GPU.

```
import tensorflow as tf
t = tf.constant([[1, 2], [3, 4]])
print(t.shape)
```

OUTPUT

```
(2, 2)
```

Shape (2, 2) means 2 rows and 2 columns.

5.3 A First Model in Keras

This trains a network to recognise **handwritten digits** (the MNIST dataset). The dataset downloads automatically.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1. load data
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# 2. scale pixels from 0-255 to 0-1
x_train = x_train / 255.0
x_test = x_test / 255.0

# 3. build the model
model = keras.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# 4. compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 5. train
model.fit(x_train, y_train, epochs=5)

# 6. test
loss, acc = model.evaluate(x_test, y_test)
print('Accuracy :', round(acc, 4))
```

OUTPUT

```
Epoch 1/5   loss: 0.2601 - accuracy: 0.9258
Epoch 5/5   loss: 0.0562 - accuracy: 0.9827
Accuracy : 0.9775
```

Reading the model line by line:

- `Flatten(input_shape=(28, 28))` : each digit image is 28×28 pixels. Flatten turns that grid into one straight line of 784 numbers.
- `Dense(128, activation='relu')` : a hidden layer of 128 neurons using ReLU.
- `Dense(10, activation='softmax')` : 10 output neurons, one per digit 0–9. Softmax turns them into probabilities.
- `optimizer='adam'` , `loss='sparse_categorical_crossentropy'` : the Adam optimiser and the cross-entropy loss for multi-class problems (see Part 4).
- The output shows loss going **down** (0.26 → 0.056) and accuracy going **up** (92.6% → 98.3%) during training, with **97.75%** on unseen test images.

Every deep learning program follows these same six steps:

1 Load

2 Prepare

3 Build

4 Compile

5 Train

6 Test

If you remember only one thing from this section, it should be this list.

Word	Meaning
Epoch	One full pass through the whole training data
Batch	A small group of images processed together, usually 32
Loss	How wrong the model is. Training tries to make this smaller.

5.4 Computer Vision in Deep Learning

Computer vision is teaching a computer to understand images. Face unlock on a phone, number plate reading, and medical scan analysis are all computer vision.

How a computer sees an image

An image is only a **grid of numbers**. In a grayscale image, each number is a brightness from **0 (black) to 255 (white)**. A colour image is three such grids stacked together: **Red, Green and Blue**.

```
import cv2
img = cv2.imread('cat.jpg')
print('Shape      ', img.shape)      # height, width, channels
print('Pixel [0,0]:', img[0, 0])     # colour values of one pixel
```

OUTPUT

```
Shape      : (480, 640, 3)
Pixel [0,0]: [ 75  98 122]
```

So a 480×640 colour photo is really **921,600 numbers** ($480 \times 640 \times 3$). The model only ever sees numbers, never a "cat".

Note: Small detail: OpenCV (`cv2`) stores colours in **Blue, Green, Red** order, so `[75 98 122]` means B=75, G=98, R=122.

The main computer vision tasks

Task	Question it answers	Output
Classification	What is in this image?	One label
Object detection	What objects, and where are they?	Boxes with labels
Segmentation	Which class does each pixel belong to?	A coloured mask

5.5 Preparing Images for a Model

Two steps are always needed before an image goes into a model.

```
# 1. resize to the size the model expects
img = cv2.resize(img, (224, 224))

# 2. scale pixel values from 0-255 to 0-1
img = img / 255.0
print(img.shape, img.min(), img.max())
```

OUTPUT

```
(224, 224, 3) 0.0 1.0
```

5.6 Data Augmentation

If we have few images, we create more by **flipping, rotating and zooming** the ones we already have. The model then sees more variety and learns better. This is called **augmentation**.

```
from tensorflow.keras import layers
augment = tf.keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2)
])
```

5.7 CNN Layers and Their Purpose

A **CNN (Convolutional Neural Network)** is the type of network used for images. It is built by stacking a few kinds of layers.

Layer	Working of layer
Convolution (Conv2D)	Finds features such as edges, shapes and textures
ReLU activation	Removes negative values so the network can learn complex things
Pooling (MaxPool)	Shrinks the image and keeps only the strongest features
Dropout	Switches off some neurons randomly to avoid overfitting
Flatten	Converts the 2-D result into a single straight line of numbers
Dense (Fully Connected)	Takes the final decision
Softmax	Converts the decision into percentages

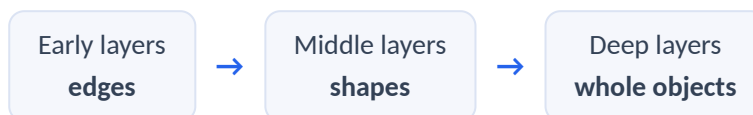
IN SIMPLE WORDS

Think of it as a factory line: convolution finds the parts, pooling keeps the important ones, and dense makes the final judgement. (**Overfitting** means the model memorises the training images instead of learning general rules, so it does badly on new images.)

What a convolution actually does

A small filter, usually **3×3**, slides over the image. At each position it multiplies and adds the pixel values underneath, producing one number. The result is a new image that highlights one kind of feature. (See the worked example in section 4.13.)

The important point: we do not design these filters. The network **learns them by itself during training**. Early layers learn edges, middle layers learn shapes, and deep layers learn whole objects like a face or a wheel.



A filter (done by hand, to see what it does)

```
import cv2
import numpy as np
gray = cv2.imread('cat.jpg', cv2.IMREAD_GRAYSCALE)
edge_filter = np.array([[-1, 0, 1],
                        [-2, 0, 2],
                        [-1, 0, 1]])
result = cv2.filter2D(gray, -1, edge_filter)
cv2.imwrite('edges.jpg', result)
```

This filter is negative on the left and positive on the right, so it lights up where brightness changes from left to right: **vertical edges**. The result is saved as edges.jpg.

A simple CNN

```
from tensorflow import keras
from tensorflow.keras import layers
model = keras.Sequential([
    layers.Conv2D(32, 3, activation='relu', input_shape=(64, 64, 3)),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(3, activation='softmax')
])
model.summary()
```

Line	Meaning
<code>Conv2D(32, 3, ...)</code>	32 filters, each 3×3, on a 64×64 colour image
<code>MaxPooling2D(2)</code>	Halves the width and height (2×2 windows)
<code>Conv2D(64, 3, ...)</code>	64 more 3×3 filters to find more complex features
<code>Flatten()</code>	Grid → straight line of numbers
<code>Dense(128, 'relu')</code>	128-neuron decision layer
<code>Dropout(0.5)</code>	Randomly switches off 50% of neurons during training
<code>Dense(3, 'softmax')</code>	3 outputs = 3 classes (e.g. cat / dog / rabbit), as percentages
<code>model.summary()</code>	Prints a table of all the layers and their sizes

5.8 Padding, Stride, Pooling and Sliding Windows

Sliding window → a small box that moves across the image step by step, left to right and top to bottom. At each stop, whatever is inside the box gets processed.

This same idea is used in two places: the convolution filter slides like this to find features, and the pooling window slides like this to shrink the image.

Padding

When a filter slides over an image, the output comes out slightly smaller, and the pixels at the border get used less than the ones in the middle. **Padding** fixes this by adding a border of zeros around the image (see the picture in section 4.14).

Setting	Meaning	Effect
<code>padding = 'valid'</code>	No padding is added	Output becomes smaller
<code>padding = 'same'</code>	Zeros are added around the edge	Output stays the same size

```
layers.Conv2D(32, 3, padding='same')    # 64x64 stays 64x64
layers.Conv2D(32, 3, padding='valid')   # 64x64 becomes 62x62
```

Why padding is used:

- It keeps the image size the same, so we can stack many layers.
- It stops information at the edges from being lost.

Stride

Stride → how many pixels the window jumps each time it moves.

- **Stride 1:** moves one pixel at a time. Slower, more detail.
- **Stride 2:** skips every other position. Faster, and the output becomes half the size.

```
layers.Conv2D(32, 3, strides=1)    # 64x64 -> 64x64 (with same padding)
layers.Conv2D(32, 3, strides=2)    # 64x64 -> 32x32
```

Output size formula

$$\text{Output} = (N + 2P - F) / S + 1$$

N = input size, F = filter size, P = padding, S = stride

Input N	Filter F	Padding P	Stride S	Output	Working
64	3	0	1	62	$(64 + 0 - 3) / 1 + 1 = 62$
64	3	1	1	64	$(64 + 2 - 3) / 1 + 1 = 64$
64	3	1	2	32	$(64 + 2 - 3) / 2 + 1 = 32.5 \rightarrow 32$

Note: The "Working" column is added. When the division isn't whole, the answer is rounded **down** (32.5 → 32).

Pooling

Pooling → shrinks the image while keeping the important information. The most common type is **max pooling**, which keeps only the largest value in each small window. (See the worked example in section 4.15.)

```
layers.MaxPooling2D(2)    # 64x64 becomes 32x32
layers.AveragePooling2D(2) # keeps the average instead of the maximum
```

Why pooling is used:

- It reduces the size, so the model runs faster.
- It keeps only the strongest features.
- It helps the model recognise an object even if it moves slightly.

5.9 ResNet-50 and YOLO

These are two famous **ready-made models**.

ResNet-50

A CNN with **50 layers**, made by **Microsoft in 2015**.

- It is already trained on millions of images and can recognise **1000 everyday objects**.
- Its special idea is the **skip connection**: a shortcut that carries information around a layer.
- Before this, very deep networks simply would not train properly. The skip connection solved that, which is why 50-layer networks became possible.

YOLO

Stands for "**You Only Look Once**".

- It is used for **object detection**: finding several objects in one image and drawing a box around each.
- Older methods checked thousands of small windows one by one, which was very slow.
- YOLO looks at the whole image **only once** and predicts all the boxes together. That is why it is fast enough for **live video**.

What YOLO gives for each object:

- A **bounding box**: the rectangle around the object.
- A **class name**: what the object is.
- A **confidence score**: how sure the model is.

ResNet vs YOLO

	ResNet-50	YOLO
Used for	Classification	Object detection
Answers	What is in the image?	What objects and where?
Output	One label	Many boxes with labels
Speed	Fast	Fast enough for live video

5.10 Annotation and Fine-tuning

Annotation

Annotation, also called **labelling**, means marking the correct answer on each training image by hand. The model learns from these answers, so **bad labels mean a bad model**. In real projects this is the most time-consuming step.

Type of annotation	What is marked	Used for
Image label	One class name for the whole image	Classification
Bounding box	A rectangle around each object	Object detection
Mask or polygon	The exact outline of the object	Segmentation

Common annotation tools

- **LabelImg**: simple offline tool for drawing boxes.
- **makesense.ai**: runs in the browser, nothing to install.
- **Roboflow**: online, and also converts between formats.
- **CVAT**: free and powerful, also supports video.

Basic rules for good annotation

- Draw the box tightly around the object.
- Label every object of that class in the image, not just the clearest one.
- Use the same spelling for a class name everywhere.
- Collect at least a few hundred images per class.

Fine-tuning

Training a model from zero needs millions of images and expensive hardware.

- Instead, we take a model that is **already trained**, such as ResNet-50, and adjust it for our own small problem.
- The logic is simple: the existing model has already learned how to see edges, shapes and textures.
- Those skills work for any image. We only need to change the **last part**, which makes the final decision.

Why we do it:

- Needs only a few hundred images instead of millions.
- Takes minutes instead of days.
- Gives much better accuracy than starting from zero.
- This is what almost every real project actually does.

IN SIMPLE WORDS

It's like hiring an experienced photographer and teaching them only what *your* products look like, instead of teaching someone to see from scratch.

5.11 Classification and Segmentation

Classification

Classification gives **one label to the whole image** ("This is a cat."). It is the simplest and most common task.

Type	Example	Last layer
Binary	Cat or dog: two options	<code>Dense(1, sigmoid)</code>
Multi-class	Cat, dog or rabbit: pick one	<code>Dense(3, softmax)</code>

Segmentation

Segmentation labels **every single pixel** instead of the whole image. The output is a **mask**: a picture where each pixel is coloured by its class.

Type	What it does
Semantic segmentation	All objects of the same class share one colour (all people = one colour)
Instance segmentation	Each individual object gets its own colour (person 1, person 2 ... all different)

Where it is used:

- **Medical scans**: outlining a tumour or an organ.
- **Self-driving cars**: separating road, footpath and people.
- **Satellite images**: marking buildings, water and farmland.
- **Photo editing**: removing the background from a portrait.

The best-known model for segmentation is **U-Net**, named after its **U shape**.

The three tasks side by side

Task	Output	Model to use	Labelling effort
Classification	One label	ResNet-50	Easy: folder names
Detection	Boxes + labels	YOLO	Medium: draw boxes
Segmentation	Pixel mask	U-Net, YOLO-seg	Hard: outline shapes

5.12 Key Points to Remember

- **TensorFlow** is the engine, **Keras** is the easy interface, **PyTorch** is the alternative.
- Every program follows **six** steps: load, prepare, build, compile, train, test.
- A **CNN** learns its own filters instead of being given them.
- **Convolution** finds features, **pooling** shrinks, **dense** decides.
- **Padding** keeps the size, **stride** controls the jump, **pooling** reduces the size.
- **ResNet-50** is for classification; **YOLO** is for detection.
- **Classification** labels the image, **detection** boxes the objects, **segmentation** labels every pixel.